

Diseño y Desarrollo de Software

LABORATORIO N° 03

Jectpack Compose



DOCENTE:

Juan José León

CURSO: **Suiyon**

**Programación en
Móviles
4to Ciclo**

LABORATORIO 03

Diseño de interfaces con Jetpack Compose: Registro de Producto

I. Logro de aprendizaje

Al finalizar el laboratorio, el estudiante construye una interfaz gráfica con Jetpack Compose que combina controles de ingreso de datos (OutlinedTextField), de acción (Button) y de visualización (Text, Card), gestionando el estado de la pantalla con remember y mutableStateOf, y aplicando el flujo profesional de trabajo con Git en equipos compartidos.

Conexión con el Lab 02: la semana pasada programaste la lógica del producto por consola (data class, cálculos, formato). Hoy le pones pantalla: los mismos datos (nombre, precio, cantidad) ahora los ingresa un usuario real, y tus herramientas de la semana 2 — toDoubleOrNull, el operador Elvis ?: y String.format("%.2f") — vuelven a escena para leer y mostrar los números.

II. Metodología: manual primero, IA después (regla 6 + 3)

Este laboratorio tiene dos partes. Parte A (en el laboratorio, SIN IA): construyes TÚ la pantalla completa en la rama main, con un mínimo de 6 commits distribuidos durante la sesión — la guía te marca cuándo hacer cada uno. Parte B (en casa, CON IA): en la rama mejora-ia usarás Gemini para agregar UNA mejora específica, con un mínimo de 3 commits que separan lo que generó la IA de lo que corregiste tú. Tu historial de commits, con sus horas, es la evidencia de tu proceso en ambas partes.

Regla de oro del curso: Gemini, ChatGPT y Copilot están PROHIBIDOS en la Parte A y en la rama main. Puedes consultar developer.android.com y a tu docente. Habrá preguntas orales sobre tu propio código.

III. Flujo de trabajo en máquina compartida

Las computadoras del laboratorio son compartidas entre secciones: lo que dejes en el disco se borra, y la sesión de GitHub del turno anterior puede seguir abierta. Por eso, TODA sesión de laboratorio empieza y termina con estos pasos:

Al llegar (checklist de inicio, ~10 min)

1. **Cierra la sesión ajena:** en Android Studio, Settings → Version Control → GitHub: si aparece una cuenta que NO es la tuya, selecciónala y quítala (-). Si haces commits con la sesión de otro, el trabajo queda a SU nombre y tu evaluación se pierde.
2. **Limpia el disco:** si en el Escritorio o en la carpeta de trabajo hay clones de otros alumnos, elimínalos.
3. **Inicia TU sesión:** agrega tu cuenta con Log In via GitHub (se autoriza en el navegador).
4. **Clona TU portafolio:** Clone Repository → pega la URL de tu repositorio (el de las carpetas Semana01 a Semana16) → Directory en la carpeta de trabajo indicada por el docente → Clone → Trust Project. La raíz se verá “vacía” (solo carpetas): es normal, los proyectos viven dentro de cada semana.

Regla del proyecto de la semana

- Dentro de la carpeta Semana03 hay un archivo marcador (lo creó GitHub al crear la carpeta). **Elimínalo** (clic derecho → Delete) antes de crear el proyecto.

- El proyecto SIEMPRE se crea DENTRO del clon: File → New → New Project → Empty Activity → en Save location escribe la ruta de tu clon + /Semana03/Lab03RegistroProducto. Si dejas la ruta por defecto del IDE, el proyecto nace FUERA del repositorio y Git no lo verá.
- **PROHIBIDO usar "Share Project on GitHub"**: crearía un repositorio nuevo separado y rompería tu portafolio. Con el repo clonado, solo Commit and Push.

Antes de irte (checklist de salida, ~5 min)

1. **Verifica en el NAVEGADOR** que tu repositorio en GitHub muestra el proyecto completo en Semana03 y tus 6 commits (URL: github.com/tu-usuario/tu-repo/commits/main).
2. **Cierra tu sesión de GitHub** en Android Studio (Settings → Version Control → GitHub → quitar tu cuenta).
3. Elimina tu clon del disco (opcional pero recomendado: la máquina igual se limpia).

LO QUE NO SUBAS, SE PIERDE. La máquina se limpia para el siguiente turno: GitHub es tu única copia real.

IV. Resultado final esperado

Tu app debe verse y comportarse como las siguientes figuras. La Figura 1 muestra la pantalla al abrir la app; la Figura 2 muestra lo que ocurre al llenar los campos y presionar AGREGAR PRODUCTO: aparece una Card con el resumen y el importe calculado (precio × cantidad, con 2 decimales).

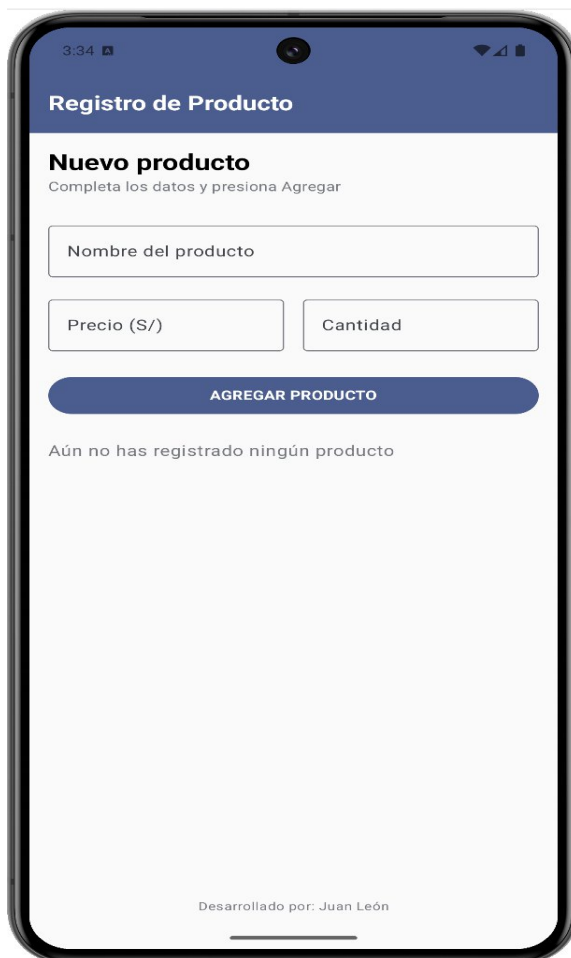


Figura 1. Pantalla inicial (formulario vacío).



Figura 2. Después de presionar AGREGAR PRODUCTO.

Detalles evaluables: precio y cantidad comparten una misma fila; el botón usa el color primario y ocupa todo el ancho; la Card usa un color de fondo suave del tema; todos los montos tienen 2 decimales.

V. Conceptos clave de Compose que usarás hoy

Concepto	¿Qué es? / ¿Para qué sirve?
@Composable	Funciones que describen la interfaz. La pantalla se construye combinando composables.
Column / Row	Organizan elementos en vertical (Column) u horizontal (Row). Son los “layouts” de Compose.
Modifier	Cadena de propiedades visuales: padding, tamaño, ancho, fondo. Ej: Modifier.fillMaxWidth().padding(16.dp).
OutlinedTextField	Control de INGRESO de datos: caja de texto con borde y etiqueta flotante (Material 3).
Button	Control de acción: ejecuta el código de su onClick al ser presionado.
Card / Text	Controles de VISUALIZACIÓN de datos: la Card agrupa información con fondo y esquinas redondeadas.
remember + mutableStateOf	El ESTADO de la pantalla. Cuando un valor de estado cambia (ej. el texto de un campo), Compose redibuja automáticamente lo necesario. Sin esto, un TextField no puede mostrar lo que escribes.

VI. Reglas de oro del diseño (se evalúan en la rúbrica)

Regla	Cómo aplicarla hoy
1. Espaciado consistente	Usa 16.dp como padding general de la pantalla y Spacer de 16.dp entre elementos. Nada debe estar pegado a los bordes.
2. Jerarquía tipográfica	Un solo título grande (MaterialTheme.typography.headlineSmall), subtítulos medianos y texto normal. No todo puede ser grande y negrita.
3. Un color primario	Usa el color primario del tema (MaterialTheme.colorScheme.primary) para los elementos principales. No mezcles 5 colores distintos.
4. Campos bien dimensionados	Campos de texto largos a ancho completo (fillMaxWidth); campos cortos como precio y cantidad comparten una fila con weight(1f).

VII. Parte A (en el laboratorio, sin IA): desarrollo paso a paso

Plan de la sesión de 2.5 horas, con los 6 commits como hitos. Si sigues las partes en orden, los commits caen solos:

Parte	Avance	Tiempo	Commit
1	Llegada, clonación y proyecto dentro de Semana03	25 min	C1
2	Encabezado con jerarquía tipográfica	20 min	C2
3	Los 3 campos con estado (fila precio/cantidad)	35 min	C3
4	Botón y Card de resumen con importe	35 min	C4
5	Pulido de diseño y mensaje de confirmación	15 min	C5
6	README con capturas + checklist de salida	20 min	C6

Parte 1: Llegada, clonación y proyecto (25 min)

1. Ejecuta el checklist de inicio de la sección III: sesión ajena fuera, tu sesión dentro, tu portafolio clonado.
2. Elimina el marcador de Semana03 y crea el proyecto **Lab03RegistroProducto** con Save location DENTRO de Semana03 (regla de la sección III). Kotlin, plantilla Empty Activity.
3. Cuando Gradle sincronice, verifica la rama **main** junto al nombre del proyecto (señal de que detectó el Git de tu portafolio). Si el IDE pregunta “Add Files to Git”: marca todo → Add. Si aparece “IDE settings can be added to Git”: Don’t Ask Again.
4. **COMMIT 1** (Git → Commit, marca los checkboxes — incluida la eliminación del marcador — y Commit and Push): “Crea proyecto Lab03 en Semana03”.

Parte 2: Encabezado con jerarquía (20 min)

1. En MainActivity.kt, elimina la función Greeting y su Preview, y crea el composable PantallaRegistro. Llámalo desde setContent (dentro del theme, pasándole el innerPadding del Scaffold como modifier, igual que hacía Greeting).

Construye el esqueleto: una Column con padding general y los textos de encabezado con jerarquía tipográfica:

```
@Composable
fun PantallaRegistro(modifier: Modifier = Modifier) {
    Column(
        modifier = modifier
            .fillMaxSize()
            .padding(16.dp)
    ) {
        Text(
            text = "Nuevo producto",
            style = MaterialTheme.typography.headlineSmall
        )
        Text(
            text = "Completa los datos y presiona Agregar",
            style = MaterialTheme.typography.bodyMedium,
            color = MaterialTheme.colorScheme.outline
        )
    }
}
```

```

        Spacer(modifier = Modifier.height(24.dp))
        // aquí irán los campos de texto
    }
}

```

2. Ejecuta y verifica los dos textos con jerarquía distinta. Los imports salen con Alt+Enter (elige siempre `androidx.compose.*`).
3. **COMMIT 2:** "Agrega encabezado con jerarquía tipográfica".

Parte 3: Controles de ingreso con estado (35 min)

Un `TextField` necesita una variable de ESTADO que guarde lo que el usuario escribe. Declara los estados al inicio de `PantallaRegistro`:

```

var nombre by remember { mutableStateOf("") }
var precio by remember { mutableStateOf("") }
var cantidad by remember { mutableStateOf("") }
var mostrarResumen by remember { mutableStateOf(false) }

```

Imports necesarios (Alt+Enter): `androidx.compose.runtime.getValue`, `setValue`, `remember`, `mutableStateOf`.

1. Agrega el primer campo dentro de la `Column` y ejecútalo. Lo que tecleas se guarda en `nombre` y la pantalla se redibuja mostrándolo. ESO es el estado:

```

OutlinedTextField(
    value = nombre,
    onChange = { nombre = it },
    label = { Text("Nombre del producto") },
    modifier = Modifier.fillMaxWidth()
)

```

2. Crea TÚ los campos de precio y cantidad con el mismo patrón, en una misma fila como en las figuras:

```

Spacer(modifier = Modifier.height(16.dp))
Row(modifier = Modifier.fillMaxWidth()) {
    OutlinedTextField(
        // TODO: estado precio, label "Precio ($)"
        modifier = Modifier.weight(1f)
    )
    Spacer(modifier = Modifier.width(16.dp))
    OutlinedTextField(
        // TODO: estado cantidad, label "Cantidad"
        modifier = Modifier.weight(1f)
    )
}

```

¿Qué hace `weight(1f)`? Reparte el ancho disponible en partes iguales entre los elementos de la `Row`.

3. **COMMIT 3:** "Agrega campos de ingreso con estado".

Parte 4: Botón y Card de resumen (35 min)

1. Agrega el botón debajo de los campos. Su `onClick` solo cambia el estado:

```

Spacer(modifier = Modifier.height(24.dp))
Button(

```

```

onClick = { mostrarResumen = true },
modifier = Modifier.fillMaxWidth()
){
    Text("AGREGAR PRODUCTO")
}

```

2. Debajo del botón, muestra la Card SOLO cuando mostrarResumen sea true. Aquí vuelven tus herramientas del Lab 02: toDoubleOrNull/toIntOrNull con el Elvis ?: (si escriben letras, la app no se cae):

```

Spacer(modifier = Modifier.height(24.dp))
if (mostrarResumen) {
    val precioNum = precio.toDoubleOrNull() ?: 0.0
    val cantidadNum = cantidad.toIntOrNull() ?: 0
    val importe = 0.0 // TODO: calcula precio x cantidad

    Card(
        modifier = Modifier.fillMaxWidth(),
        colors = CardDefaults.cardColors(
            containerColor = MaterialTheme.colorScheme.primaryContainer
        )
    ){
        Column(modifier = Modifier.padding(16.dp)) {
            Text(nombre, style = MaterialTheme.typography.titleLarge)
            Text("Precio: S/ " + String.format("%.2f", precioNum))
            // TODO: Text de cantidad
            // TODO: Text de importe en negrita y color primario
        }
    }
}

```

3. Completa los TODO y verifica contra la Figura 2: nombre, precio, cantidad e importe con 2 decimales.
4. **COMMIT 4:** "Agrega boton y card de resumen con importe calculado".

Parte 5: Pulido de diseño (15 min)

1. Repasa la tabla de Reglas de oro y verifica tu pantalla contra las figuras: espaciados de 16.dp, jerarquía de textos, color primario.
2. Agrega el mensaje verde "✓ Producto registrado correctamente" debajo de la Card (Text con color = Color(0xFF2E7D32)).
3. **COMMIT 5:** "Aplica reglas de diseño y mensaje de confirmación".

Parte 6: README y salida (20 min)

1. Toma 2 capturas del emulador: pantalla vacía y pantalla con producto registrado (como las Figuras 1 y 2).
2. Crea o edita el README.md dentro de la carpeta del proyecto: título, tu nombre, descripción, las 2 capturas, y responde: ¿qué pasaría si declaras las variables de los campos SIN remember? (pruébalo antes de responder).
3. **COMMIT 6:** "Agrega README con capturas y respuesta sobre remember". Commit and Push.
4. Ejecuta el checklist de salida de la sección III: verifica tus 6 commits en el navegador, cierra tu sesión de GitHub, limpia la máquina. Envía la URL de tu repositorio al docente.

VIII. Parte B (en casa, con IA): rama mejora-ia (3 puntos)

Con tu Parte A completa en GitHub, harás tu primera mejora asistida por IA desde tu propia computadora. Si es la primera vez en esa máquina, clona tu portafolio (una sola vez); si ya lo tienes clonado, abre el proyecto y haz Git → Pull para traer lo del laboratorio.

1. **1. Crea la rama:** en Android Studio, menú Git → New Branch → nómbrala mejora-ia (se crea a partir de tu main terminada). Todo lo que hagas ahora vive en esa rama; main no se toca.
2. **2. La mejora (única y específica):** pídele a Gemini que agregue validación de campos vacíos (si falta un dato al presionar AGREGAR, mostrar un mensaje de error en rojo en lugar de la Card) y un botón Limpiar que vacíe el formulario. Sé específico en tu prompt: dónde (PantallaRegistro), qué (el comportamiento exacto) y qué NO tocar.
3. **COMMIT B1:** “Aplica mejora generada con IA: validacion y boton limpiar” — el código tal como lo generó Gemini (después de verificar que compila y corre).
4. **3. Revisa críticamente:** prueba los casos (campos vacíos, precio con letras) y lee el código hasta entenderlo. Ajusta lo que no te convenza: nombres, textos, espaciados, lógica.
5. **COMMIT B2:** “Corrige codigo de la IA: (qué ajustaste y por qué)” — TUS correcciones sobre lo generado. Separar B1 de B2 hace visible en el diff qué escribió la máquina y qué corregiste tú: eso es exactamente lo que se evalúa.
6. **4. Documenta en el README** una sección “Mejora con IA” con una tabla de 3 columnas: Prompt que usé | Qué generó Gemini | Qué acepté o corregí (y por qué).
7. **COMMIT B3:** “Documenta prompts y decisiones en README”. Push de la rama (Android Studio ofrecerá publicarla en GitHub: acepta).

Regla de oro: Gemini SOLO se usa dentro de la rama mejora-ia. El docente comparará ambas ramas en GitHub (selector de ramas del repositorio) y revisará las horas de tus commits.

IX. Entregables

1. **URL de tu repositorio (portafolio)** enviada al finalizar la sesión de laboratorio.
2. Rama main: proyecto completo en Semana03, mínimo 6 commits descriptivos con horas dentro de la sesión, README con 2 capturas y la respuesta sobre remember.
3. Rama mejora-ia (durante la semana): mejora funcionando, mínimo 3 commits (B1 aplicación, B2 corrección, B3 documentación) y tabla de prompts en el README.

X. Rúbrica de evaluación (20 puntos)

Criterio	Descripción	Puntaje
Estructura de la pantalla	Column principal con título, textos de encabezado y organización igual a las figuras.	3
Controles de ingreso	Los 3 OutlinedTextField funcionan (se puede escribir y el texto se ve), con etiquetas correctas y precio/cantidad en una misma fila.	4
Botón y Card de resumen	Al presionar AGREGAR PRODUCTO aparece la Card con nombre, precio, cantidad e importe calculado con 2 decimales.	4
Reglas de diseño	Cumple las 4 reglas de oro: espaciado 16.dp, jerarquía tipográfica, color primario único, campos bien dimensionados.	2
GitHub rama main	Mínimo 6 commits descriptivos con horas DENTRO de la sesión de laboratorio, y README con las 2 capturas y la respuesta sobre remember.	4

Parte B: Mejora con IA	Rama mejora-ia con validación y botón Limpiar funcionando, mínimo 3 commits (aplicación, corrección, documentación) y tabla de prompts en el README.	3
TOTAL		20

Nota: commits de la rama main con horas fuera del horario del laboratorio, o un único commit gigante al final, pierden los puntos del criterio “GitHub rama main”. El proceso importa tanto como el resultado.

XI. Preguntas de reflexión (para la defensa oral)

- ¿Qué es el estado en Compose y por qué un TextField lo necesita?
- ¿Qué hace remember? ¿Qué observaste cuando lo quitaste?
- ¿Cuál es la diferencia entre un control de ingreso y uno de visualización? Da un ejemplo de cada uno en tu pantalla.
- ¿Qué hace Modifier.weight(1f) en la fila de precio y cantidad?
- ¿Para qué usaste toDoubleOrNull y el operador ?: en tu código? ¿De qué laboratorio vienen?
- Muestra tu historial de commits de main y explica el avance de cada uno.
- Parte B: ¿qué le pediste a Gemini, qué generó, y qué corregiste tú? Muéstralo en el diff de tus commits B1 y B2.

Docente : Juan León